STAT 639 - Homework 3

ISLR Chapter 5: Exercises 3, 6; Chapter 6: Exercises 2, 3, 9(a)-(d); Chapter 8: Exercises 4, 5

Bao Dang

October 23, 2026

Contents

1 Chapter 5 - Exercise 3 **3**
1.1 (a) How K-fold cross-validation is implemented . . . 3
1.2 (b) Advantages and disadvantages . . . 3
1.2.1 (i) Compared with the validation set approach . . . 3
1.2.2 (ii) Compared with leave-one-out cross-validation (LOOCV) . . . 3

2 Chapter 5 - Exercise 6 **4**
2.1 (a) Standard errors from logistic regression . . . 4
2.2 (b) Define a bootstrap function . . . 4
2.3 (c) Estimate standard errors with the bootstrap . . . 4
2.4 (d) Compare the results . . . 5

3 Chapter 6 - Exercise 2 **6**
3.1 (a) Lasso relative to least squares . . . 6
3.2 (b) Ridge regression relative to least squares . . . 6
3.3 (c) Nonlinear methods relative to least squares . . . 6
3.4 Summary . . . 6

4 Chapter 6 - Exercise 3 **7**
4.1 (a) Training RSS . . . 7
4.2 (b) Test RSS . . . 7
4.3 (c) Variance . . . 7
4.4 (d) Squared bias . . . 7
4.5 (e) Irreducible error . . . 7
4.6 Summary . . . 7

5 Chapter 6 - Exercise 9(a)-(d) **9**
5.1 (a) Training and test split . . . 9
5.2 (b) Least squares regression . . . 9
5.3 (c) Ridge regression with cross-validation . . . 9
5.4 (d) Lasso regression with cross-validation . . . 11
5.5 Comparison of the three methods . . . 12

6 Chapter 8 - Exercise 4 **14**
6.1 (a) Tree corresponding to the left-hand partition . . . 14
6.2 (b) Partition corresponding to the right-hand tree . . . 16

7 Chapter 8 - Exercise 5 **18**
7.1 Majority voting . . . 18

7.2 Average probability 18
7.3 Reproducible calculation in R 18
7.4 Interpretation 19

1 Chapter 5 - Exercise 3

This exercise considers K**-fold cross-validation** and compares it with the validation set approach and leave-one-out cross-validation (LOOCV).

1.1 (a) How K-fold cross-validation is implemented

First, randomly divide the n observations into K approximately equal-sized, non-overlapping folds. For each $k = 1, \ldots, K$, fit the model using the observations in the other $K - 1$ folds and evaluate its prediction error on the held-out kth fold. Repeat until each fold has served as the validation fold exactly once.

For regression, if the folds are equally sized, the resulting cross-validation estimate of test mean squared error is

$$CV_{(K)} = \frac{1}{K} \sum_{k=1}^{K} MSE_k,$$

where MSE_k is the mean squared prediction error on the kth validation fold. More generally, when folds have different sizes n_k, the average is weighted by fold size:

$$CV_{(K)} = \sum_{k=1}^{K} \frac{n_k}{n} MSE_k.$$

For classification, the same procedure can use misclassification error instead of mean squared error.

1.2 (b) Advantages and disadvantages

1.2.1 (i) Compared with the validation set approach

Advantages. K-fold cross-validation generally provides a more stable estimate of test error because every observation is used for validation once, rather than relying on a single random training-validation split. With a common 50/50 validation split, K-fold CV with $K = 5$ or $K = 10$ also fits models using a larger fraction of the available observations. This generally reduces the bias of the estimated test error relative to the error of a model trained on the full dataset.

Disadvantages. K-fold cross-validation is more computationally expensive: it requires fitting the model K times, whereas a single validation split requires only one fit. The improvement in stability is not an absolute guarantee for every dataset or split.

1.2.2 (ii) Compared with leave-one-out cross-validation (LOOCV)

Advantages. K-fold CV with $K = 5$ or $K = 10$ is usually less computationally expensive than LOOCV, which fits the model n times. It also often has lower variance as an estimate of test error. LOOCV training sets overlap almost completely, so the resulting validation errors can be strongly correlated; averaging them does not necessarily reduce variance as much as expected.

Disadvantages. K-fold CV generally has higher bias than LOOCV when estimating the test error of a model fitted to the full dataset. Each K-fold model is trained on only $(K - 1)n/K$ observations, while each LOOCV model is trained on $n - 1$ observations. The LOOCV training size is therefore closer to the full sample size, and its test-error estimate typically has less bias. Unlike LOOCV, the result of K-fold CV can also depend on the random assignment of observations to folds.

For example, with $n = 100$, five-fold CV trains each model on 80 observations, while LOOCV trains each model on 99 observations. The comparison illustrates the **bias-variance trade-off in estimating test error**: LOOCV generally has lower bias, whereas five- or ten-fold CV often has lower variance. This concerns the bias and variance of the *test-error estimator*, not directly those of the fitted regression coefficients.

2 Chapter 5 - Exercise 6

We use the `Default` data set from `ISLR2` to compare model-based standard errors with bootstrap standard errors for a logistic regression of `default` on `income` and `balance`. The model is

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 \,\text{income} + \beta_2 \,\text{balance},$$

where $p = P(\text{default} = \text{Yes})$.

2.1 (a) Standard errors from logistic regression

```
# Install packages once if needed: install.packages(c("ISLR2", "boot"))
library(ISLR2)
library(boot)

data(Default)
set.seed(639)
fit <- glm(default ~ income + balance,
           data = Default, family = binomial)

# Coefficient estimates and model-based standard errors
summary(fit)$coefficients[c("income", "balance"),
                          c("Estimate", "Std. Error")]
```

```
##             Estimate   Std. Error
## income  2.080898e-05 4.985167e-06
## balance 5.647103e-03 2.273731e-04
```

The `Std. Error` column reports model-based standard errors calculated from the fitted logistic regression. These quantify the estimated sampling uncertainty in each coefficient.

2.2 (b) Define a bootstrap function

The bootstrap function receives the data and a vector of sampled row indices. It fits the same logistic regression to those rows and returns the two coefficient estimates.

```
boot.fn <- function(data, index) {
  model <- glm(default ~ income + balance,
               data = data[index, , drop = FALSE],
               family = binomial)
  coef(model)[c("income", "balance")]
}

# Check that the function returns the original-data estimates
boot.fn(Default, seq_len(nrow(Default)))
```

```
##       income      balance
## 2.080898e-05 5.647103e-03
```

2.3 (c) Estimate standard errors with the bootstrap

We generate $B = 1000$ bootstrap samples by sampling observations with replacement, refit the logistic model for each sample, and compute the standard deviation of the resulting coefficient estimates.

```r
set.seed(639)
boot.out <- boot(data = Default, statistic = boot.fn, R = 1000)
boot.out
```

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = Default, statistic = boot.fn, R = 1000)
##
##
## Bootstrap Statistics :
##         original        bias     std. error
## t1* 2.080898e-05 -1.432634e-07 4.675857e-06
## t2* 5.647103e-03  8.214820e-06 2.237298e-04
```

```r
# Compare model-based and bootstrap standard errors
model.se <- summary(fit)$coefficients[
  c("income", "balance"), "Std. Error"]
bootstrap.se <- apply(boot.out$t, 2, sd)

se.comparison <- data.frame(
  Predictor = c("income", "balance"),
  GLM_SE = unname(model.se),
  Bootstrap_SE = unname(bootstrap.se)
)
knitr::kable(se.comparison, digits = 7,
             caption = "Model-based versus bootstrap standard errors")
```

Table 1: Model-based versus bootstrap standard errors

Predictor	GLM_SE	Bootstrap_SE
income	0.0000050	0.0000047
balance	0.0002274	0.0002237

2.4 (d) Compare the results

The table above compares the standard errors estimated by `glm()` with the bootstrap standard errors. Both methods estimate sampling uncertainty in the income and balance coefficients. They should generally be reasonably close if the fitted model and bootstrap approximation are well behaved, but they need not be identical because they rely on different estimation procedures and the bootstrap result has Monte Carlo variation. After running the code, comment on the observed differences for **each predictor**, using the values in the table.

3 Chapter 6 - Exercise 2

This exercise compares the flexibility and expected prediction accuracy of the lasso, ridge regression, and nonlinear methods relative to ordinary least squares (OLS). The relevant bias–variance decomposition is

$$\mathrm{E}\left[(Y - \hat{f}(X))^2\right] = \mathrm{Var}(\hat{f}(X)) + \mathrm{Bias}(\hat{f}(X))^2 + \sigma^2.$$

Here, σ^2 is the irreducible error. A method can improve expected test prediction accuracy even if it increases one component of the error, provided the other component decreases by a greater amount.

3.1 (a) Lasso relative to least squares

Answer: (iii). The lasso is less flexible than OLS because it minimizes a penalized residual sum of squares:

$$\min_{\beta_0, \boldsymbol{\beta}} \left\{ \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^{p} |\beta_j| \right\}.$$

The L_1 penalty shrinks coefficients toward zero and may set some exactly to zero. Relative to unpenalized least squares, this restriction generally increases squared bias but reduces variance. It can improve prediction accuracy when the **decrease in variance exceeds the increase in squared bias**. Therefore, option (iii) is correct.

3.2 (b) Ridge regression relative to least squares

Answer: (iii). Ridge regression is also less flexible than OLS. Its objective is

$$\min_{\beta_0, \boldsymbol{\beta}} \left\{ \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^{p} \beta_j^2 \right\}.$$

The L_2 penalty shrinks coefficients toward zero, usually without setting them exactly to zero. The restriction typically increases squared bias while reducing variance. Ridge regression improves expected test prediction accuracy when its reduction in variance outweighs the increase in squared bias. Thus, option (iii) is correct.

3.3 (c) Nonlinear methods relative to least squares

Answer: (ii). In the comparison intended here, nonlinear methods are more flexible than a linear least-squares model. They can represent nonlinear relationships and therefore may reduce squared bias, but their greater flexibility can increase variance. They improve expected test prediction accuracy when the **decrease in squared bias exceeds the increase in variance**. Hence, option (ii) is correct.

3.4 Summary

Part	Method	Flexibility relative to OLS	Correct option
(a)	Lasso	Less flexible	(iii)
(b)	Ridge regression	Less flexible	(iii)
(c)	Nonlinear methods	More flexible (in this comparison)	(ii)

Reference: James et al., *An Introduction to Statistical Learning*, 2nd ed., Chapter 6, Exercise 2; Section 6.2.

4 Chapter 6 - Exercise 3

We consider the lasso regression estimator obtained by minimizing the residual sum of squares (RSS), subject to an L_1 constraint:

$$\min_{\beta_0,\ldots,\beta_p} \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 \quad \text{subject to} \quad \sum_{j=1}^{p} |\beta_j| \leq s.$$

The parameter $s \geq 0$ controls the size of the feasible coefficient set. When $s = 0$, all slope coefficients must be zero, giving an intercept-only model. Increasing s relaxes the constraint, allowing a more flexible fit. This is the opposite direction from increasing the penalty parameter λ in the penalized lasso formulation.

4.1 (a) Training RSS

Answer: (iv), steadily decrease. Increasing s expands the set of feasible coefficient vectors. A coefficient vector feasible at a smaller value of s remains feasible at a larger value, so the minimum achievable training RSS cannot increase. More precisely, training RSS is **non-increasing** and can plateau when the constraint is no longer active.

4.2 (b) Test RSS

Answer: (ii), decrease initially and then increase (U-shaped). With s near zero, the intercept-only or heavily constrained model tends to underfit, producing high bias. Relaxing the constraint initially reduces bias and can lower test RSS. At sufficiently large s, the increase in variance can outweigh the reduction in squared bias, raising test RSS. This U shape is the **typical bias-variance pattern**, not a guarantee for every dataset.

4.3 (c) Variance

Answer: (iii), steadily increase. As s increases, the model becomes more flexible, and its fitted predictions can respond more strongly to differences among training samples. Prediction variance therefore generally increases as the constraint is relaxed. This is a typical qualitative trend rather than a strict monotonicity theorem for every data-generating process.

4.4 (d) Squared bias

Answer: (iv), steadily decrease. At small s, strong shrinkage can prevent the model from representing important predictor effects. Allowing larger coefficient magnitudes generally reduces the systematic error introduced by the constraint, so squared bias tends to decrease as s increases. This is the standard qualitative bias-variance picture.

4.5 (e) Irreducible error

Answer: (v), remain constant. Irreducible error is the noise inherent in the data-generating process and cannot be eliminated by changing the model's flexibility. Therefore, it does not depend on s.

4.6 Summary

Part	Quantity	Choice	Behavior as s increases
(a)	Training RSS	(iv)	Non-increasing
(b)	Test RSS	(ii)	Typically U-shaped
(c)	Variance	(iii)	Generally increases

Part	Quantity	Choice	Behavior as s increases
(d)	Squared bias	(iv)	Generally decreases
(e)	Irreducible error	(v)	Constant

The expected test error can be understood through the bias-variance decomposition:

$$\mathrm{E}\left[(Y - \hat{f}(X))^2\right] = \mathrm{Bias}(\hat{f}(X))^2 + \mathrm{Var}(\hat{f}(X)) + \sigma^2.$$

Reference: James et al., *An Introduction to Statistical Learning*, 2nd ed., Chapter 6, Exercise 3; Sections 2.2.2 and 6.2.2.

5 Chapter 6 - Exercise 9(a)-(d)

We predict the number of applications received by a college (`Apps`) using the remaining variables in the `College` dataset. We compare ordinary least squares, ridge regression, and lasso on the **same held-out test set**. Ridge and lasso tuning parameters are selected by 10-fold cross-validation using **training data only**.

5.1 (a) Training and test split

We randomly allocate 70% of the observations to training and the remaining 30% to testing. The random seed makes the split reproducible.

```
if (!requireNamespace("ISLR2", quietly = TRUE) ||
    !requireNamespace("glmnet", quietly = TRUE)) {
  stop("Install required packages first: install.packages(c('ISLR2', 'glmnet'))")
}
library(ISLR2)
library(glmnet)
data("College", package = "ISLR2")

set.seed(639)
train_id <- sample(seq_len(nrow(College)), size = floor(0.70 * nrow(College)))
train <- College[train_id, , drop = FALSE]
test  <- College[-train_id, , drop = FALSE]

knitr::kable(data.frame(
  Partition = c("Training", "Test", "Total"),
  Observations = c(nrow(train), nrow(test), nrow(College))
), caption = "Training/test split of the College dataset")
```

Table 2: Training/test split of the College dataset

Partition	Observations
Training	543
Test	234
Total	777

5.2 (b) Least squares regression

We fit a multiple linear regression with all other variables as predictors. Test MSE is the mean squared difference between observed and predicted applications on the test set.

```
lm_fit <- lm(Apps ~ ., data = train)
lm_pred <- predict(lm_fit, newdata = test)
lm_mse <- mean((test$Apps - lm_pred)^2)
lm_mse
```

```
## [1] 926010.9
```

The least-squares model has a test MSE of **926,010.8** applications squared.

5.3 (c) Ridge regression with cross-validation

Ridge regression minimizes RSS plus an L_2 penalty, $\lambda \sum_{j=1}^{p} \beta_j^2$. We use `model.matrix()` to encode categorical predictors consistently and `cv.glmnet()` with `alpha = 0` to choose λ using training-set cross-validation. The intercept is excluded from the penalized coefficient matrix; `glmnet` handles it separately.

```r
x_train <- model.matrix(Apps ~ ., data = train)[, -1, drop = FALSE]
x_test  <- model.matrix(Apps ~ ., data = test)[, -1, drop = FALSE]
y_train <- train$Apps
y_test  <- test$Apps
stopifnot(identical(colnames(x_train), colnames(x_test)))
```

```r
set.seed(639)
ridge_cv <- cv.glmnet(x_train, y_train, alpha = 0, nfolds = 10)
ridge_lambda <- ridge_cv$lambda.min
ridge_pred <- as.numeric(predict(ridge_cv, newx = x_test, s = "lambda.min"))
ridge_mse <- mean((y_test - ridge_pred)^2)
data.frame(lambda_min = ridge_lambda, test_MSE = ridge_mse)
```

```
##   lambda_min test_MSE
## 1   380.0948 877429.2
```

```r
plot(ridge_cv)
```

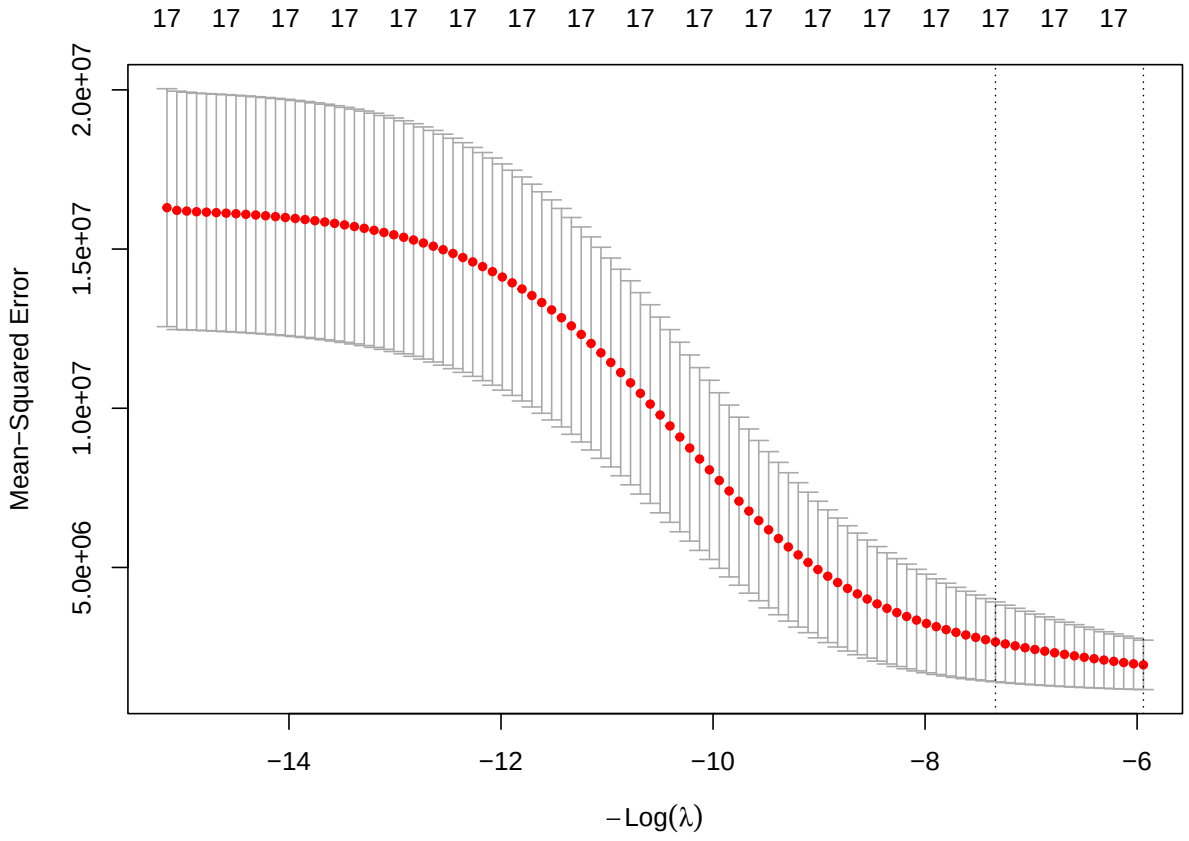


Figure 1: Ridge regression: cross-validation error by penalty size

Cross-validation selects $\lambda =$ **380.1** . The ridge test MSE is **877,429.2** applications squared.

5.4 (d) Lasso regression with cross-validation

Lasso uses an L_1 penalty, $\lambda \sum_{j=1}^{p} |\beta_j|$, which can shrink some predictor coefficients to exactly zero. We again select λ using only the training data.

```
set.seed(639)
lasso_cv <- cv.glmnet(x_train, y_train, alpha = 1, nfolds = 10)
lasso_lambda <- lasso_cv$lambda.min
lasso_pred <- as.numeric(predict(lasso_cv, newx = x_test, s = "lambda.min"))
lasso_mse <- mean((y_test - lasso_pred)^2)

lasso_coef <- as.matrix(coef(lasso_cv, s = "lambda.min"))
nonzero_count <- sum(lasso_coef[-1, 1] != 0)  # Excludes intercept

knitr::kable(data.frame(
  Selected_lambda = signif(lasso_lambda, 5),
  Test_MSE = round(lasso_mse, 2),
  Nonzero_predictor_coefficients = nonzero_count
), caption = "Lasso test error and number of selected predictor coefficients")
```

Table 3: Lasso test error and number of selected predictor coefficients

Selected_lambda	Test_MSE	Nonzero_predictor_coefficients
2.2262	923521.3	17

```
plot(lasso_cv)
```

```
selected <- data.frame(
  Predictor = rownames(lasso_coef)[-1],
  Coefficient = as.numeric(lasso_coef[-1, 1])
)
selected <- selected[selected$Coefficient != 0, , drop = FALSE]
knitr::kable(selected, digits = 4,
             caption = "Predictors retained by lasso (intercept excluded)")
```

Table 4: Predictors retained by lasso (intercept excluded)

Predictor	Coefficient
PrivateYes	-459.6784
Accept	1.6300
Enroll	-0.8089
Top10perc	46.7805
Top25perc	-9.8013
F.Undergrad	0.0249
P.Undergrad	0.0378
Outstate	-0.0977
Room.Board	0.1255
Books	0.0447
Personal	0.0823
PhD	-10.4638
Terminal	-1.5277
S.F.Ratio	28.7451

Predictor	Coefficient
perc.alumni	-0.8188
Expend	0.1052
Grad.Rate	6.4454

Lasso selects $\lambda =$ **2.226** and obtains a test MSE of **923,521.3** applications squared. It retains **17** nonzero predictor coefficients (excluding the intercept).

5.5 Comparison of the three methods

```
comparison <- data.frame(
  Method = c("Least squares", "Ridge", "Lasso"),
  Test_MSE = c(lm_mse, ridge_mse, lasso_mse),
  Test_RMSE = sqrt(c(lm_mse, ridge_mse, lasso_mse))
)
knitr::kable(comparison, digits = 2,
             caption = "Held-out test prediction error across methods")
```

Table 5: Held-out test prediction error across methods

Method	Test_MSE	Test_RMSE
Least squares	926010.8	962.29
Ridge	877429.2	936.71
Lasso	923521.3	961.00

Among the three approaches, **Ridge** achieves the lowest test MSE for this particular split. RMSE is reported in applications and is therefore easier to interpret than MSE, which is in squared applications. Differences should be interpreted with care: a single random split is not enough to establish that one method is universally superior.

Reference: James et al., *An Introduction to Statistical Learning*, 2nd ed., Chapter 6, Exercise 9(a)–(d); Sections 6.2.1–6.2.2.

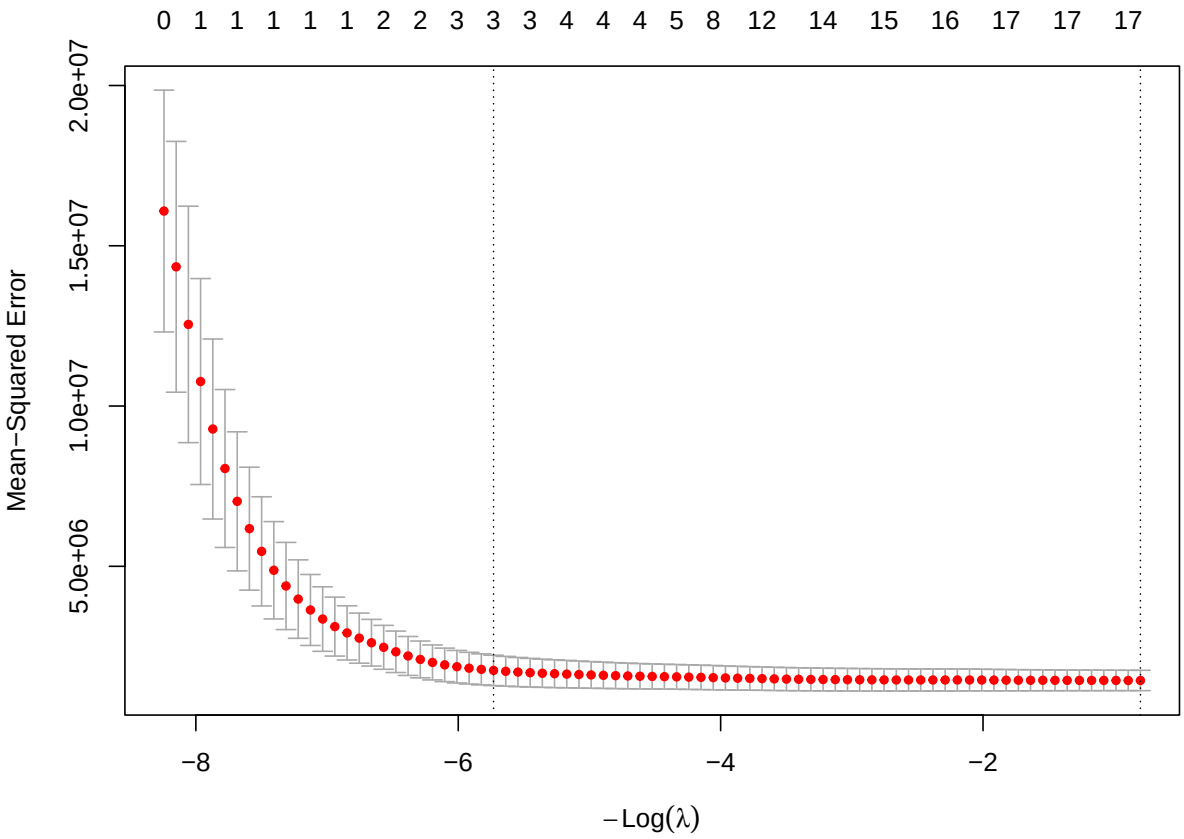


Figure 2: Lasso regression: cross-validation error by penalty size

6 Chapter 8 - Exercise 4

Figure 8.14 illustrates two equivalent representations of a regression tree: a partition of the (X_1, X_2) predictor space and a tree diagram. Each terminal region predicts the mean of Y in that region.

6.1 (a) Tree corresponding to the left-hand partition

The full-height boundary at $X_1 = 1$ gives the root split. For $X_1 < 1$, the next split is $X_2 = 1$. Below that boundary, split at $X_1 = 0$; within $0 \leq X_1 < 1$ and $X_2 < 1$, split again at $X_2 = 0$. The five terminal predictions are 5, 15, 3, 0, and 10.

```
plot(NA, xlim=c(0,10), ylim=c(0,8), axes=FALSE,
     xlab="", ylab="", main="(a) Tree reconstructed from predictor partition")
node <- function(x,y,label) {
  rect(x-.63,y-.28,x+.63,y+.28,col="gray95",border="gray45")
  text(x,y,label,cex=.88)
}
leaf <- function(x,y,label) {
  rect(x-.52,y-.26,x+.52,y+.26,col="white",border="gray45")
  text(x,y,label,cex=.9,font=2)
}
edge <- function(x1,y1,x2,y2,label="") {
  segments(x1,y1-.29,x2,y2+.29)
  if(nchar(label)>0) text((x1+x2)/2,(y1+y2)/2+.18,label,cex=.75)
}
node(5,7.25,expression(X[1]<1))
node(3.5,5.6,expression(X[2]<1))
leaf(8.0,5.6,"5")
edge(5,7.25,3.5,5.6,"Yes")
edge(5,7.25,8,5.6,"No")
node(4.8,4.0,expression(X[1]<0))
leaf(1.4,4.0,"15")
edge(3.5,5.6,4.8,4.0,"Yes")
edge(3.5,5.6,1.4,4.0,"No")
leaf(3.4,2.4,"3")
node(6.3,2.4,expression(X[2]<0))
edge(4.8,4,3.4,2.4,"Yes")
edge(4.8,4,6.3,2.4,"No")
leaf(5.1,.8,"10")
leaf(8,.8,"0")
edge(6.3,2.4,5.1,.8,"Yes")
edge(6.3,2.4,8,.8,"No")
```

The resulting terminal regions are:

Conditions	Predicted mean of Y
$X_1 \geq 1$	5
$X_1 < 1,\ X_2 \geq 1$	15
$X_1 < 0,\ X_2 < 1$	3
$0 \leq X_1 < 1,\ 0 \leq X_2 < 1$	0
$0 \leq X_1 < 1,\ X_2 < 0$	10

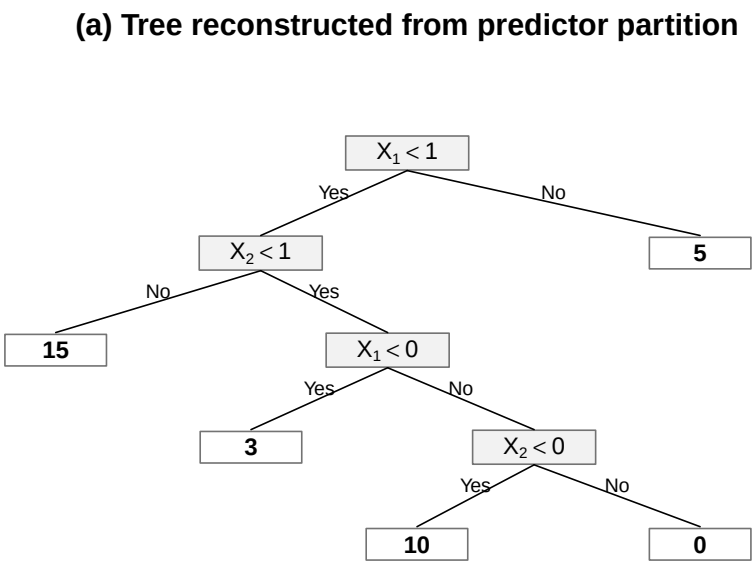


Figure 3: Regression tree reconstructed from the left panel of ISLR Figure 8.14

6.2 (b) Partition corresponding to the right-hand tree

The tree first divides the plane at $X_2 = 1$. Below that boundary, the split at $X_1 = 1$ gives predictions -1.80 (left) and 0.63 (right). Above $X_2 = 1$, the tree divides at $X_2 = 2$. The top region has prediction 2.49; the middle band $1 \leq X_2 < 2$ is divided at $X_1 = 0$, giving predictions -1.06 (left) and 0.21 (right).

```
plot(NA, xlim=c(-2,2), ylim=c(-.5,3), xlab=expression(X[1]),
     ylab=expression(X[2]), xaxs="i", yaxs="i",
     main="(b) Predictor-space partition")
rect(-2,-.5,2,3,border="black")
abline(h=1,lwd=2)
segments(-2,2,2,2,lwd=2)
segments(1,-.5,1,1,lwd=2)
segments(0,1,0,2,lwd=2)
text(-.5,.2,"-1.80",cex=1.05)
text(1.5,.2,"0.63",cex=1.05)
text(-1,1.5,"-1.06",cex=1.05)
text(1,1.5,"0.21",cex=1.05)
text(0,2.5,"2.49",cex=1.05)
```

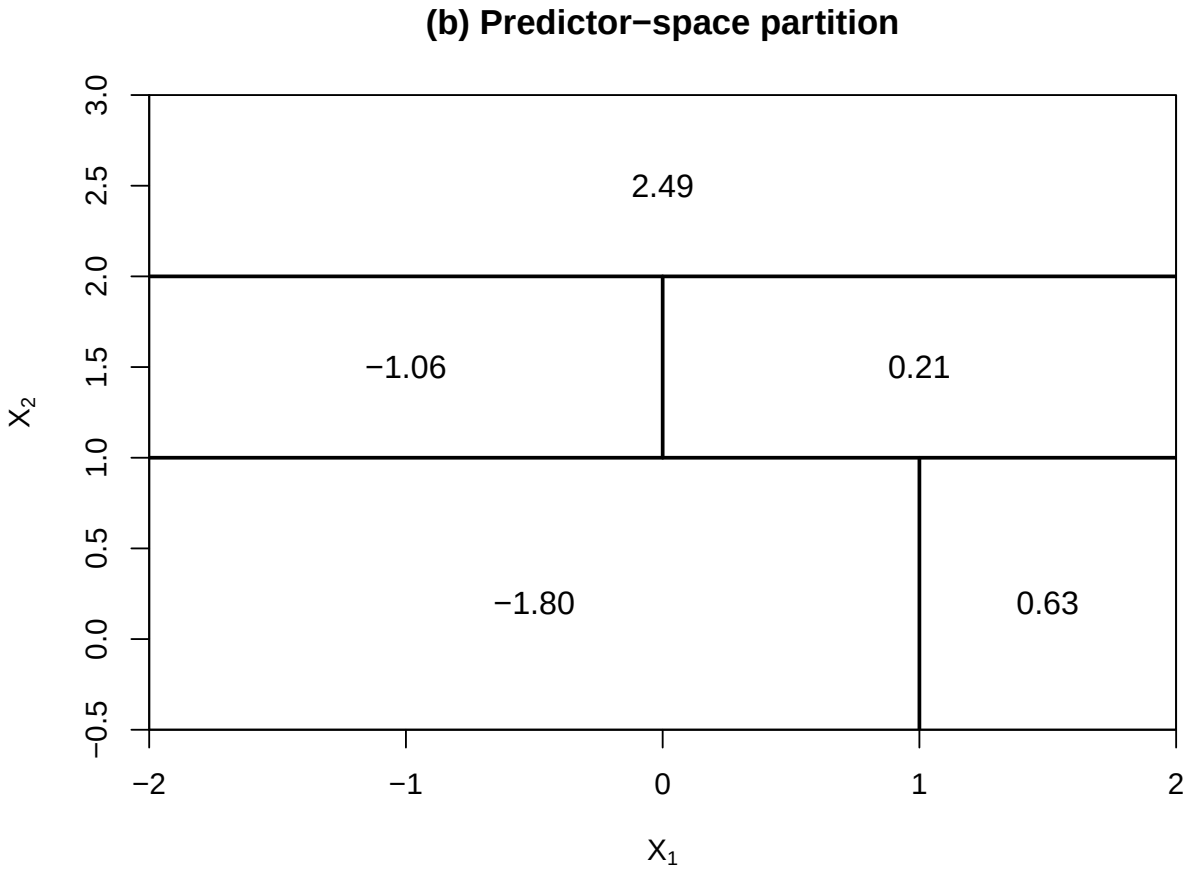


Figure 4: Predictor-space partition reconstructed from the right panel of ISLR Figure 8.14

The resulting terminal regions are:

Conditions	Predicted mean of Y
$X_2 < 1,\ X_1 < 1$	-1.80
$X_2 < 1,\ X_1 \geq 1$	0.63
$1 \leq X_2 < 2,\ X_1 < 0$	-1.06
$1 \leq X_2 < 2,\ X_1 \geq 0$	0.21
$X_2 \geq 2$	2.49

Interpretation. Each internal node creates a binary split; following the path to a terminal node identifies a rectangular region of predictor space. The terminal value is the fitted mean response for all observations in that region.

Reference: James et al., *An Introduction to Statistical Learning*, 2nd ed., Chapter 8, Exercise 4, Figure 8.14.

7 Chapter 8 - Exercise 5

The exercise considers ten classification trees fitted to bootstrap samples. For one predictor value X, the estimated probabilities of the **Red** class are

$$0.10,\ 0.15,\ 0.20,\ 0.20,\ 0.55,\ 0.60,\ 0.60,\ 0.65,\ 0.70,\ 0.75.$$

We compare majority voting with classification based on the average estimated probability.

7.1 Majority voting

Under majority voting, each tree first predicts a class. Using the standard 0.5 classification threshold, a tree predicts **Red** when its estimated probability of Red is greater than 0.5 and **Green** otherwise.

Four trees predict Green (probabilities 0.10, 0.15, 0.20, and 0.20), while six trees predict Red (probabilities 0.55, 0.60, 0.60, 0.65, 0.70, and 0.75). Thus, the **majority-vote classification is Red**.

7.2 Average probability

The average estimated probability of Red is

$$\overline{p}_{\text{Red}} = \frac{0.10 + 0.15 + 0.20 + 0.20 + 0.55 + 0.60 + 0.60 + 0.65 + 0.70 + 0.75}{10} = \frac{4.50}{10} = 0.45.$$

Because $0.45 < 0.50$, the **average-probability classification is Green**.

7.3 Reproducible calculation in R

```
red_prob <- c(0.10, 0.15, 0.20, 0.20, 0.55,
              0.60, 0.60, 0.65, 0.70, 0.75)

individual_votes <- ifelse(red_prob > 0.5, "Red", "Green")
vote_counts <- table(factor(individual_votes, levels = c("Red", "Green")))
majority_class <- names(which.max(vote_counts))

mean_red_prob <- mean(red_prob)
average_class <- ifelse(mean_red_prob > 0.5, "Red", "Green")

knitr::kable(
  data.frame(
    Tree = seq_along(red_prob),
    Probability_Red = red_prob,
    Individual_Vote = individual_votes
  ),
  caption = "Estimated red-class probabilities and individual tree votes"
)
```

Table 6: Estimated red-class probabilities and individual tree votes

Tree	Probability_Red	Individual_Vote
1	0.10	Green
2	0.15	Green
3	0.20	Green
4	0.20	Green

Tree	Probability_Red	Individual_Vote
5	0.55	Red
6	0.60	Red
7	0.60	Red
8	0.65	Red
9	0.70	Red
10	0.75	Red

```
knitr::kable(
  data.frame(
    Method = c("Majority voting", "Average probability"),
    Result = c(
      paste0(vote_counts[["Red"]], " Red votes; ", vote_counts[["Green"]], " Green votes"),
      sprintf("Mean P(Red | X) = %.2f", mean_red_prob)
    ),
    Final_Class = c(majority_class, average_class)
  ),
  caption = "Comparison of bagging aggregation rules"
)
```

Table 7: Comparison of bagging aggregation rules

Method	Result	Final_Class
Majority voting	6 Red votes; 4 Green votes	Red
Average probability	Mean P(Red | X) = 0.45	Green

7.4 Interpretation

The methods disagree because majority voting gives each tree's **class label** one vote, regardless of its estimated probability, whereas averaging probabilities preserves the strength of the estimated class probabilities. Although six trees predict Red, the four low Red probabilities reduce the mean to 0.45. Therefore, the final predictions are **Red under majority voting** and **Green under average probability**.

Reference: James et al., *An Introduction to Statistical Learning*, 2nd ed., Chapter 8, Exercise 5.